

在本代码中，以房价预测为例子，

列举一些通用的数据处理方法和机器学习算法。

注意：所有必要的python库需要提前安装才可以运行，

例如pip install -i <https://pypi.tuna.tsinghua.edu.cn/simple> xgboost

是安装xgboost库的命令

```
In [152... import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [153... from sklearn.pipeline import Pipeline, FeatureUnion
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

读取csv格式的表格数据

```
In [174... data=pd.read_csv('housing.csv')
```

备份数据（在处理数据之前，备份原始数据集是一个非常好的习惯，

这可以防止因数据清洗或转换过程中的错误而导致的数据丢失。）

```
In [155... # 保存数据备份
data.to_csv('housing_backup.csv', index=False)
# 使用copy()方法创建data的一个拷贝
data_copy = data.copy()
```

显示数据前5行

```
In [156... data.head()
```

```
Out[156... 
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
0	-122.23	37.88	41.0	880.0	129.0	322.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0

显示数据前8行

```
In [157... data.head(8)
```

Out[157...

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
0	-122.23	37.88	41.0	880.0	129.0	322.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0
5	-122.25	37.85	52.0	919.0	213.0	413.0
6	-122.25	37.84	52.0	2535.0	489.0	1094.0
7	-122.25	37.84	52.0	3104.0	687.0	1157.0

显示数据后5行

In [158...

```
data.tail()
```

Out[158...

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
20635	-121.09	39.48	25.0	1665.0	374.0	8
20636	-121.21	39.49	18.0	697.0	150.0	3
20637	-121.22	39.43	17.0	2254.0	485.0	10
20638	-121.32	39.43	18.0	1860.0	409.0	7
20639	-121.24	39.37	16.0	2785.0	616.0	13

显示数据后6行

In [159...

```
data.tail(6)
```

Out[159...

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
20634	-121.56	39.27	28.0	2332.0	395.0	10
20635	-121.09	39.48	25.0	1665.0	374.0	8
20636	-121.21	39.49	18.0	697.0	150.0	3
20637	-121.22	39.43	17.0	2254.0	485.0	10
20638	-121.32	39.43	18.0	1860.0	409.0	7
20639	-121.24	39.37	16.0	2785.0	616.0	13

显示数据行列信息

In [160...

```
data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  float64
3   total_rooms            20640 non-null  float64
4   total_bedrooms         20433 non-null  float64
5   population             20640 non-null  float64
6   households             20640 non-null  float64
7   median_income          20640 non-null  float64
8   median_house_value     20640 non-null  float64
9   ocean_proximity        20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB

```

单独查看某一列数据

```

In [161...] data['median_house_value'].head(3)

Out[161...] 0    452600.0
            1    358500.0
            2    352100.0
            Name: median_house_value, dtype: float64

```

统计每一种靠海情况的数量

```

In [162...] data['ocean_proximity'].value_counts()

Out[162...] ocean_proximity
<1H OCEAN      9136
INLAND         6551
NEAR OCEAN     2658
NEAR BAY       2290
ISLAND          5
            Name: count, dtype: int64

```

描述数据统计信息，包括数量统计、均值、方差、最小值、最大值、中位数等

```

In [163...] data.describe()

```

Out[163...

	longitude	latitude	housing_median_age	total_rooms	total_bedroom
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.87055
std	2.003532	2.135952	12.585558	2181.615252	421.38507
min	-124.350000	32.540000	1.000000	2.000000	1.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000

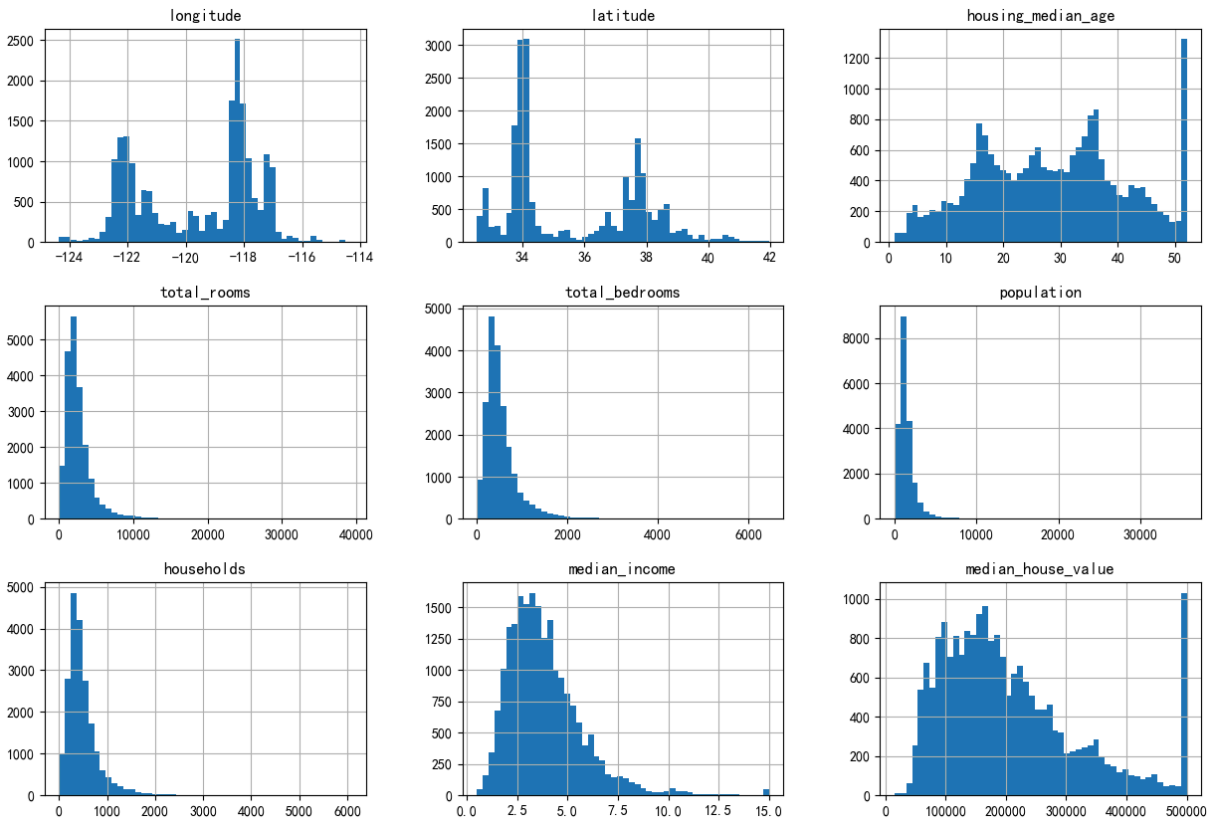
画出全部数据的统计直方图

In [164...

```
data.hist(bins=50,figsize=(15,10))
```

Out[164...

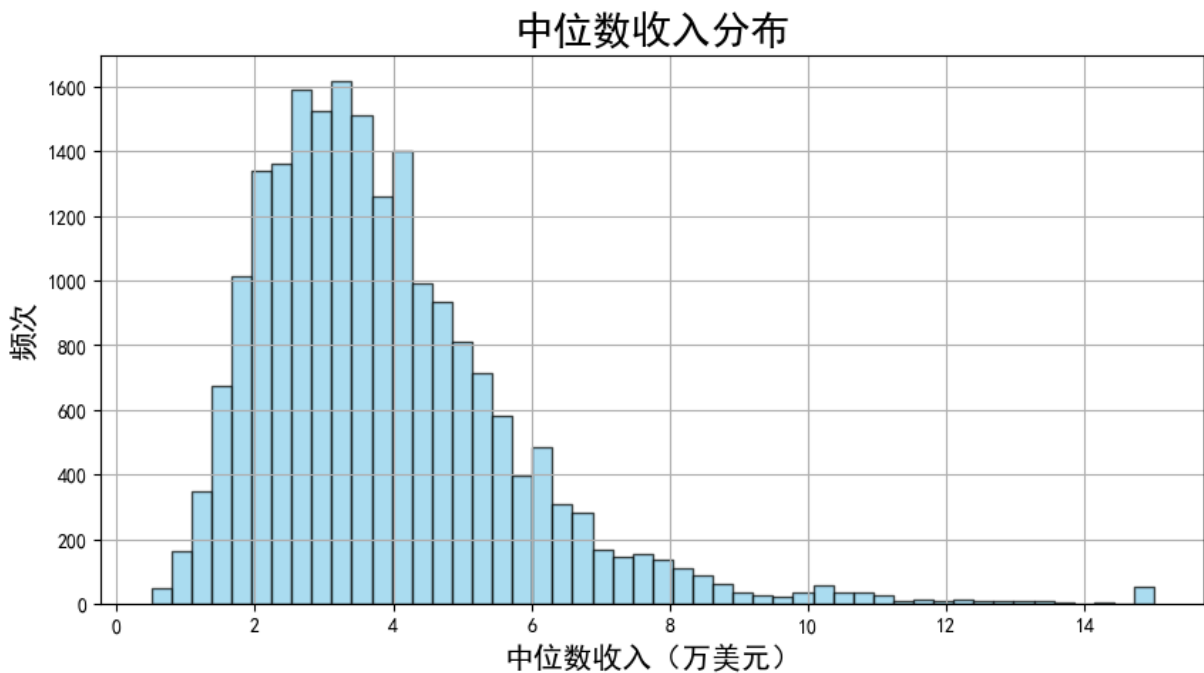
```
array([[<Axes: title={'center': 'longitude'}>,  
       <Axes: title={'center': 'latitude'}>,  
       <Axes: title={'center': 'housing_median_age'}>],  
      [<Axes: title={'center': 'total_rooms'}>,  
       <Axes: title={'center': 'total_bedrooms'}>,  
       <Axes: title={'center': 'population'}>],  
      [<Axes: title={'center': 'households'}>,  
       <Axes: title={'center': 'median_income'}>,  
       <Axes: title={'center': 'median_house_value'}>]], dtype=object)
```



选择median_income这一列画出直方图,

并设置绘制直方图时的一些参数 (这部分可以仅作了解)

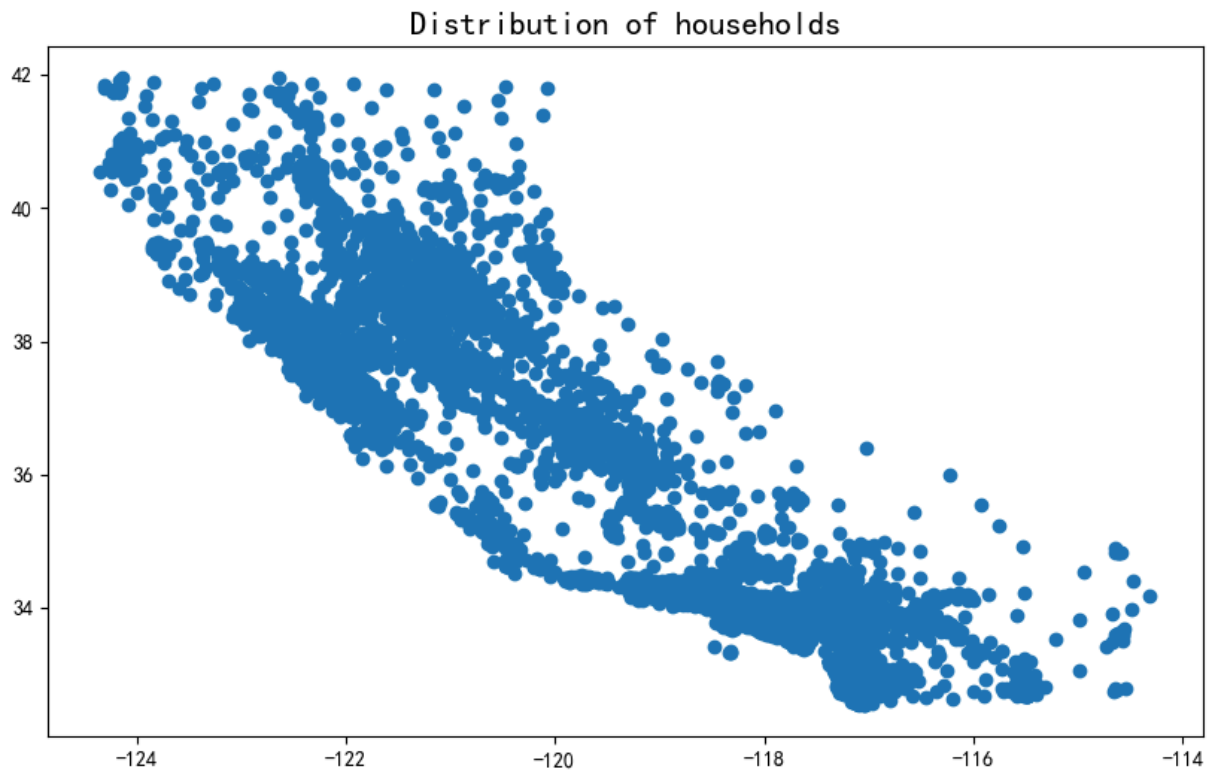
```
In [165... data["median_income"].hist(  
    bins=50,                # bins的数量  
    figsize=(10, 5),        # 图片尺寸  
    color='skyblue',        # 柱子颜色  
    edgecolor='black',      # 柱子边缘颜色  
    alpha=0.7               # 透明度 (0 = 透明, 1 = 不透明)  
)  
  
# 设置支持中文的字体 (确保该字体在您的系统上可用)  
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签  
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号  
  
# 添加标题和标签  
plt.title("中位数收入分布", fontsize=20)  
plt.xlabel("中位数收入 (万美元)", fontsize=15)  
plt.ylabel("频次", fontsize=15)  
  
# 显示图表  
plt.show()
```



选择任意两列数据, 例如: 经度和纬度, 绘制二维散点图

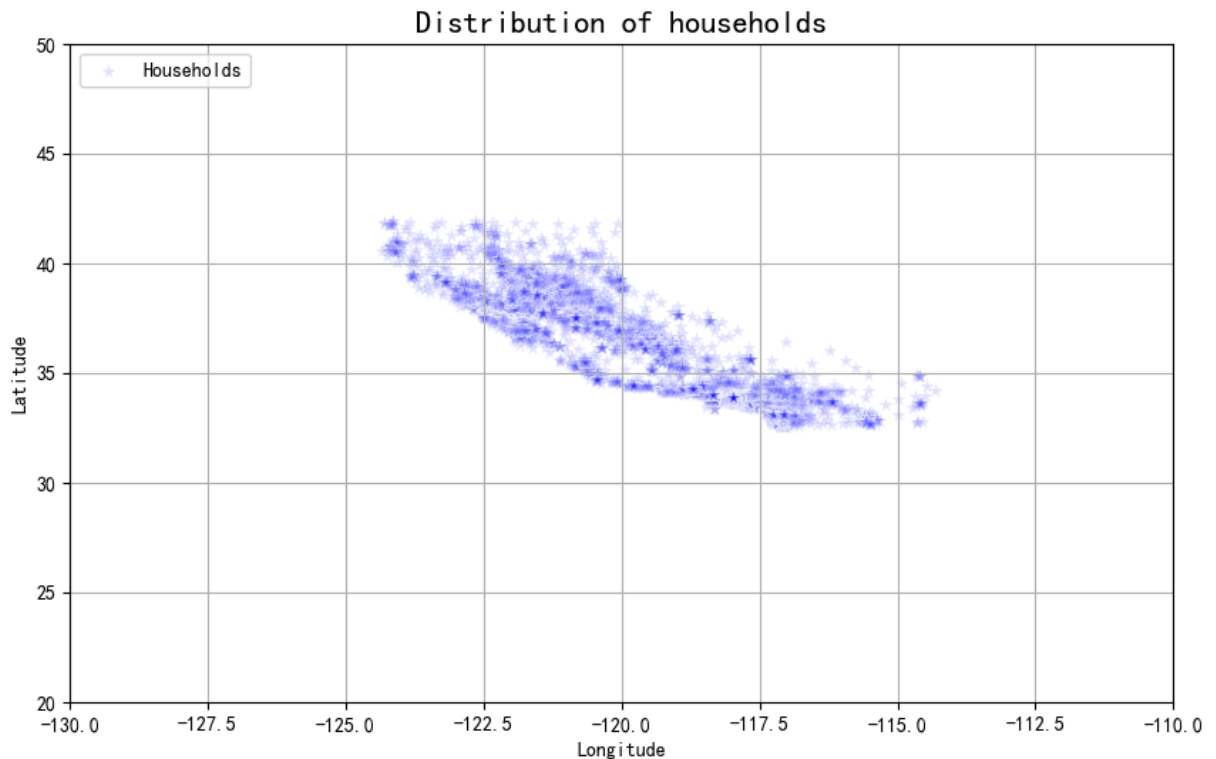
```
In [166... plt.figure(figsize=(10,6))  
plt.scatter(x=data['longitude'],y=data['latitude'])  
plt.title("Distribution of households",size=16)
```

```
Out[166... Text(0.5, 1.0, 'Distribution of households')
```



设置散点图的一些参数

```
In [167... plt.figure(figsize=(10, 6))
scatter =
plt.scatter(x=data['longitude'], y=data['latitude'], s=50, c='blue',
            alpha=0.1, edgecolor='white', linewidth=1, marker='*',
            label='Households')
plt.xlim(-130, -110) # 设置x轴范围 (经度范围)
plt.ylim(20, 50)     # 设置y轴范围 (纬度范围)
plt.title("Distribution of households", size=16)
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.legend(handles=[scatter], loc='upper left') # 显示图例
plt.grid(True) # 显示网格线 (可选)
plt.show()
```



循环检查每一列数据是否有空值

```
In [168... for i in data.columns:
              print(i,': ',data[i].isnull().sum())
```

```
longitude : 0
latitude : 0
housing_median_age : 0
total_rooms : 0
total_bedrooms : 207
population : 0
households : 0
median_income : 0
median_house_value : 0
ocean_proximity : 0
```

处理空值 (缺失值)

```
In [178... # 删除包含空值的行
df_cleaned = data.dropna()
```

```
In [179... # 用列的平均值填充空值 (按列计算)
df_filled = data["total_bedrooms"].fillna(data["total_bedrooms"].mean())
```

特征编码: 将ocean_proximity转换为数值

```
In [180... from sklearn.preprocessing import LabelEncoder, StandardScaler
label_encoder = LabelEncoder()
df_cleaned.loc[:, 'ocean_proximity'] =
label_encoder.fit_transform(df_cleaned['ocean_proximity'])
```

特征缩放：标准化所有数值特征（除了已经编码的分类特征）

```
In [181... # 假设median_house_value是目标变量
numeric_features =
df_cleaned.columns.drop(['ocean_proximity', 'median_house_value'])
scaler = StandardScaler()
df_cleaned.loc[:, numeric_features] =
scaler.fit_transform(df_cleaned[numeric_features])
```

如果只取一部分特征和标签，例如，

只选择housing_median_age、households、median_income这三个特征，

和median_house_value这个标签，那么可以用以下方法：

```
In [182... data_selected =
data[['housing_median_age', 'households', 'median_income', 'median_house_value']]
data_selected.head(3)
```

```
Out[182...      housing_median_age  households  median_income  median_house_value
0                41.0         126.0          8.3252         452600.0
1                21.0        1138.0          8.3014         358500.0
2                52.0         177.0          7.2574         352100.0
```

划分训练集和测试集

```
In [183... from sklearn.model_selection import train_test_split
# 分离特征和目标变量
X = df_cleaned.drop('median_house_value', axis=1)
y = df_cleaned['median_house_value']
# 按照8:2的比例划分训练集和测试集
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2, random_state=42)

# 随机状态 (random_state) 是一个用于随机数生成的种子。
# 设置一个固定的种子可以确保每次运行代码时都得到相同的结果，便于复现。

# 现在X_train, X_test, y_train, y_test分别包含了训练集和测试集的特征和目标变量。

# 如果需要将结果保存到新的CSV文件中，可以使用以下代码：
# 训练集
X_train.to_csv('train_features.csv', index=False)
y_train.to_csv('train_labels.csv', index=False, header=['median_house_value'])
# 测试集
X_test.to_csv('test_features.csv', index=False)
y_test.to_csv('test_labels.csv', index=False, header=['median_house_value'])

# 注意：将y_train和y_test保存到CSV时，由于它们现在是Series对象，
# 我们需要指定header参数来添加列名，否则CSV文件中将只有数据没有列名。
# 同时，index=False参数用于避免将DataFrame的索引也保存到CSV中。
```


选择机器学习算法

```
In [184... # 导入评估指标的库
from sklearn.metrics import mean_squared_error, r2_score
```

线性回归

```
In [197... # 导入库
from sklearn.linear_model import LinearRegression
# 初始化线性回归模型
lin_reg = LinearRegression()
# 训练线性回归模型
lin_reg.fit(X_train, y_train)
# 用线性回归模型预测房价
y_pred_lin = lin_reg.predict(X_test)
print("房价预测: ", y_pred_lin)
# 评估线性回归模型
mse_lin = mean_squared_error(y_test, y_pred_lin)
rmse_lin = mse_lin ** 0.5
r2_lin = r2_score(y_test, y_pred_lin)
print(f"Linear Regression - MSE: {mse_lin}, RMSE: {rmse_lin}, R²: {r2_lin}")
```

房价预测: [195854.43379086 157564.349841 202476.8302992 ... 131125.6167362
155283.141129 165053.13670181]

Linear Regression - MSE: 4924108937.911958, RMSE: 70171.99539639697, R²: 0.639923667
9243399

随机森林回归

```
In [198... # 导入库
from sklearn.ensemble import RandomForestRegressor
# 初始化
rf_reg = RandomForestRegressor(n_estimators=100, random_state=42)
# 训练
rf_reg.fit(X_train, y_train)
# 预测
y_pred_rf = rf_reg.predict(X_test)
print("房价预测: ", y_pred_rf)
# 评估随机森林回归模型
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = mse_rf ** 0.5
r2_rf = r2_score(y_test, y_pred_rf)
print(f"Random Forest Regressor - MSE: {mse_rf}, RMSE: {rmse_rf}, R²: {r2_rf}")
```

房价预测: [235237.05 155161. 194334. ... 90603. 111460. 119159.]

Random Forest Regressor - MSE: 2450800681.307609, RMSE: 49505.562124953285, R²: 0.82
07847691631327

K近邻回归

```
In [202... from sklearn.neighbors import KNeighborsRegressor
# 可以调整n_neighbors参数, 例如n_neighbors=7, 具体看题目是否给出K的值
```

```
knn_reg = KNeighborsRegressor(n_neighbors=5)
knn_reg.fit(X_train, y_train)
y_pred_knn = knn_reg.predict(X_test)
mse_knn = mean_squared_error(y_test, y_pred_knn)
r2_knn = r2_score(y_test, y_pred_knn)
print(f"K-Nearest Neighbors Regressor - MSE: {mse_knn}, R²: {r2_knn}")
```

K-Nearest Neighbors Regressor - MSE: 3869393341.7594223, R²: 0.7170499313832146

支持向量机回归

```
In [204... from sklearn.svm import SVR
# 可以调整kernel, C, gamma, epsilon参数
svr_reg = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.1)
svr_reg.fit(X_train, y_train)
y_pred_svr = svr_reg.predict(X_test)
mse_svr = mean_squared_error(y_test, y_pred_svr)
r2_svr = r2_score(y_test, y_pred_svr)
print(f"Support Vector Regressor - MSE: {mse_svr}, R²: {r2_svr}")
```

Support Vector Regressor - MSE: 9553762596.999563, R²: 0.3013794298978312

岭回归

```
In [211... from sklearn.linear_model import Ridge
# alpha是正则化强度的超参数, 可以根据需要调整
ridge_reg = Ridge(alpha=0.5, random_state=42)
ridge_reg.fit(X_train, y_train)
y_pred_ridge = ridge_reg.predict(X_test)
# 计算均方误差 (MSE) 和决定系数 (R²)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)
print(f"Ridge Regressor - MSE: {mse_ridge}, R²: {r2_ridge}")
```

Ridge Regressor - MSE: 4924130132.6269455, R²: 0.6399221180570789

决策树

```
In [214... from sklearn.tree import DecisionTreeRegressor
# 可以调整max_depth, min_samples_split等参数
dtr_reg = DecisionTreeRegressor(random_state=42)
dtr_reg.fit(X_train, y_train)
y_pred_dtr = dtr_reg.predict(X_test)
# 计算MSE和R²
mse_dtr = mean_squared_error(y_test, y_pred_dtr)
r2_dtr = r2_score(y_test, y_pred_dtr)
print(f"Decision Tree - MSE: {mse_dtr}, R²: {r2_dtr}")
```

Decision Tree - MSE: 4696208506.1335945, R²: 0.6565889270784173

梯度提升回归

```
In [215... from sklearn.ensemble import GradientBoostingRegressor
# 可以调整n_estimators, learning_rate, max_depth等参数
```

```

gbr_reg = GradientBoostingRegressor(random_state=42)
gbr_reg.fit(X_train, y_train)
y_pred_gbr = gbr_reg.predict(X_test)
# 计算MSE和R2
mse_gbr = mean_squared_error(y_test, y_pred_gbr)
r2_gbr = r2_score(y_test, y_pred_gbr)
print(f"Gradient Boosting - MSE: {mse_gbr}, R2: {r2_gbr}")

```

Gradient Boosting - MSE: 3193512140.3214154, R²: 0.7664738631039225

XGBoost回归

In [220...

```

import xgboost as xgb
# 可以调整n_estimators, learning_rate, max_depth等参数
xgb_reg = xgb.XGBRegressor(objective='reg:squarederror', random_state=42)
xgb_reg.fit(X_train, y_train)
y_pred_xgb = xgb_reg.predict(X_test)
# 计算MSE和R2
# 计算MSE和R2
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
print(f"xgboost - MSE: {mse_xgb}, R2: {r2_xgb}")

```

xgboost - MSE: 2346502318.353185, R²: 0.8284116052968699